Laborator 2

Exercitiul 1 -> poza 7.png

Use the following code to generate a regression dataset:

```
X, y = make_regression(
    n_samples=1000,
    n_features=1,
    noise=2,
    bias=3,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 20% of the initial dataset. Apply the ridge regression algorithm with λ = 0.2 on the train set. Compute w, w0, and the mean squared error on the test set.

```
(array([16.72745169]), 2.981569554579661, 4.315212551575999)
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets     # for make_regression function
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```
X, y = datasets.make_regression(
    n_samples=1000,
    n_features=1,
    noise=2,
    bias=3,
    random_state=42,
)
```

```
# in cazul nostru -> data este X si target ul este y
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state =
```

```
# modificam alpha din parametrii la 0.2
ridge_reg = Ridge(alpha = 0.2, random_state = 42)
# in enunt zice sa aplicam pe train set deci X_train si y_train
ridge_reg.fit(X_train, y_train)
```

```
▾ Ridge
Ridge(alpha=0.2, random_state=42)
```

```
print("w0 : ", ridge_reg.intercept_)
print("w : ", ridge_reg.coef_)
```

```
w0 :  2.981569854879661
w :  [16.73745169]
```

```
# cand folosim cele de test -> vrem sa testam ceva
# deci de accea vom da predict pe cele de test si nu pe cele de train
y_pred = ridge_reg.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error on test set: ", mse)
```

```
Mean Squared Error on test set:  4.315312551578999
```

Exercitiul 2 -> poza 11..jpg

Use the following code to generate a regression dataset:

```
X, y = make_regression(
    n_samples=1000,
    n_features=1,
    noise=0.5,
    bias=3,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 15% of the initial dataset. Apply the stochastic gradient descent regression algorithm with ℓ2 regularization with λ = 0.01, for 150 epochs, with a constant learning rate η = 0.005 on the train set. Compute w, w0, and the mean squared error on the test set.

```
(array([16.5730716]), array([2.96446862]), 0.2826888309401727)
```

```
from sklearn.datasets import make_regression  # for make_regression function
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import SGDRegressor
```

```
# avem datasets aici
X, y = make_regression(
    n_samples=1000,
    n_features=1,
    noise=0.5,
    bias=3,
    random_state=42,
)
```

```
# data = x, target = y si test_size = 15%
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.15,
                                                    random_state = 42)
```

```
LAMBDA = 0.01
num_epochs = 150
```

```
ETA = 0.005
```

```
sgd_reg = SGDRegressor(
                        max_iter = num_epochs,
                        alpha = LAMBDA,
                        tol = 1e-3,
                        penalty = 'l2',
                        eta0 = ETA,
                        learning_rate='constant',
                        random_state=42)

# dam fit pe seturile de antrenare
sgd_reg.fit(X_train, y_train)
```

```
SGDRegressor
SGDRegressor(alpha=0.01, eta0=0.005, learning_rate='constant', max_iter=150,
             random_state=42)
```

```
print("w : ", sgd_reg.coef_)
print("w0 : ", sgd_reg.intercept_)
```

```
w :  [16.5730716]
w0 :  [2.96446862]
```

```
y_pred = sgd_reg.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("MSE : ", mse)
```

```
MSE :  0.2826888309401727
```

Exercitiul 3 -> pdf

1. Use the following code to generate a regression dataset:

```
X, y = make_regression(
    n_samples=1000,
    n_features=1,
    noise=1,
    bias=2,
    random_state=42
)
```

Split the data into a train and test split, the test split being 20% of the initial dataset. Apply the stochastic gradient descent regression algorithm with no regularization for 100 epochs with a constant learning rate η = 0.005 on the train set. Compute w, w0, and the mean squared error on the test set.

```
(array([16.74232235]), array([1.98795322]), 1.0791838748150104)
```

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
```

```
# datasets

X, y = make_regression(
    n_samples=1000,
    n_features=1,
    noise=1,
    bias=2,
    random_state=42
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.2,
                                                    random_state = 42)
```

```
EPOCHS = 100
ETA0 = 0.005
```

```
sgd_reg = SGDRegressor(
    max_iter = EPOCHS,
    tol = 1e-3,
    penalty = None,
    eta0 = ETA0,
    learning_rate = 'constant',
    random_state = 42)

# ne zice in enunt ca trb aplicate pe train set
sgd_reg.fit(X_train, y_train)
```

```
SGDRegressor
SGDRegressor(eta0=0.005, learning_rate='constant', max_iter=100, penalty=None,
             random_state=42)
```

```
y_pred = sgd_reg.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
```

```
print("w : ", sgd_reg.coef_)
print("w0 : ", sgd_reg.intercept_)
print("MSE : ", mse)
```

```
w :  [16.80725709]
w0 :  [1.95767208]
MSE :  1.093251003082448
```

Exercitiul 4 - > poza whatsapp

Use the following code to generate a regression dataset:

```
X, y = make_regression(
    n_samples=1000,
    n_features=1,
    noise=0.1,
    bias=1,
```

```
    random_state=42,
)
```

Split the data into a train and test split, the test split being 20% of the initial dataset. Apply the lasso regression algorithm with λ = 0.01 on the train set. Compute w, w0, and the mean squared error on the test set.

```
(array([16.73813334]), 0.9998273193495221, 0.010476633888361446)
```

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
```

```
X, y = make_regression(
    n_samples=1000,
    n_features=1,
    noise=0.1,
    bias=1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.2,
                                                    random_state = 42)
```

```
ALPHA = 0.01
```

```
# Both SGDRegressor with penalty='l1' and Lasso implement Lasso regression.
# This cell uses SGDRegressor.

sgd_reg = SGDRegressor(
    penalty = "l1",
    alpha = ALPHA,
    max_iter = 1000,
    tol = 1e-3,
    random_state = 42)
sgd_reg.fit(X_train, y_train)
```

```
▾ SGDRegressor
SGDRegressor(alpha=0.01, penalty='l1', random_state=42)
```

```
y_pred = sgd_reg.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
```

```
print("w : ", sgd_reg.coef_)
print("w0 : ", sgd_reg.intercept_)
print("MSE : ", mse)
```

```
w :  [16.73848775]
w0 :  [0.99884556]
MSE :  0.010813471342312853
```

```
# Metoda 2
from sklearn.linear_model import Lasso
```

```
lasso_reg = Lasso(alpha = ALPHA)
lasso_reg.fit(X_train, y_train)
y_new_reg = lasso_reg.predict(X_test)

mse = mean_squared_error(y_test, y_new_reg)

print("w : ", lasso_reg.coef_)
print("w0 : ", lasso_reg.intercept_)
print("MSE : ", mse)
```

```
w :  [16.7372794]
w0 :  0.9992424232803128
MSE :  0.010823732052063684
```

Laborator 3

Exercitiul 1 -> 7.jpg

Use the following code to generate a regression dataset:

```
X, y = make_regression(
    n_samples=1000,
    n_features = 2,
    noise=1,
    bias=2,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset. Apply the polynomial kernel SVM algorithm with degree 2, coefficient 0.5, C = 2 and epsilon=1 for 5000 epochs on the train set. Compute the second support vector and the mean squared error on the test set.

```
(array([0.11426769, 0.15010176]), 0.9811148401064861)
```

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error
```

```
X, y = make_regression(
    n_samples = 1000,
    n_features = 2,
    noise = 1,
    bias = 2,
    random_state = 42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
```

```
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
DEGREE =  2
COEFFICIENT = 0.5
EPSILON = 1
EPOCHS = 5000
```

```
svm_reg = SVR(kernel = "poly",
              degree = DEGREE,
              C = 2,
              epsilon = EPSILON,
              coef0 = COEFFICIENT,
              max_iter = EPOCHS
              )
```

```
svm_reg.fit(X_train, y_train)
```

```
SVR
SVR(C=2, coef0=0.5, degree=2, epsilon=1, kernel='poly', max_iter=5000)
```

```
print("Al doilea suport vector : ", svm_reg.support_vectors_[1])
```

```
Al doilea suport vector :  [0.11422765 0.15030176]
```

```
y_pred = svm_reg.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("MSE : ", mse)
```

```
MSE :  0.9818168401064468
```

Exercitiul 2 -> poza 10.jpg

Use the following code to generate a classification dataset:

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the polynomial kernel SVM algorithm with degree 3, coefficient 1, C = 10 for 10000 epochs on the train set.

Compute the third support vector, the accuracy on the test set, and the output probability distribution for the 9th sample in the test set.

```
(array([-1.69489734, -1.34942283, 0.1541163, 0.55650619]), 0.9, array([[0.4477966, 0.3628446,
```

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
DEGREE = 3
COEFFICIENT = 1
hyperparamC = 10
EPOCHS = 10000
```

```
poly_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel = "poly",
                        degree = DEGREE,
                        coef0 = COEFFICIENT,
                        C = hyperparamC,
                        max_iter = EPOCHS,
                        probability=True))
    ])

poly_kernel_svm_clf.fit(X_train, y_train)
```

```
▸ Pipeline
  ▸ StandardScaler
  ▸ SVC
```

```
# al treilea support vector
print("Al treilea suport vector : ", poly_kernel_svm_clf[1].support_vectors_[2])
```

```
Al treilea suport vector :  [-0.64597123 -0.38253777 -0.14682131  0.58908964]
```

```
# accuracy
accuracy = poly_kernel_svm_clf.score(X_test, y_test)
```

```
print("Accuracy : ", accuracy)
```

```
Accuracy :  0.908
```

```
#output probability distribution fot the 9th sample in the test set
y_pred = poly_kernel_svm_clf[1].predict_proba(X_test)
print("Output probability distribution for the 9th sample in the test set : ", y_pred[8])
```

```
Output probability distribution for the 9th sample in the test set :  [0.47714173 0.5170912
```

Exercitiul 3 -> 2025

Use the following code to generate a regression dataset:

```
X, y = make_regression(
    n_samples=1000,
    n_features=2,
    noise=2,
    bias=1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the Gaussian RBF kernel SVM algorithm with γ = 0.0005, C = 1000, and ε = 1 for 4000 epochs on the train set.

Compute the third support vector and the mean squared error on the test set.

```
(array([1.15811087, 0.79166269]), 3.9410634511101237)
```

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error
```

```
X, y = make_regression(
    n_samples=1000,
    n_features=2,
    noise=2,
    bias=1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
GAMMA = 0.0005
hyperparameterC = 1000
EPSILON = 1
EPOCHS = 4000
```

```
svm_rbf = SVR(kernel = "rbf",
              gamma = GAMMA,
              C = hyperparameterC,
              epsilon = EPSILON,
              max_iter = EPOCHS
              )

svm_rbf.fit(X_train, y_train)
```

```
SVR
SVR(C=1000, epsilon=1, gamma=0.0005, max_iter=4000)
```

```
print("Al treilea suport vector : ", svm_rbf.support_vectors_[2])
```

```
Al treilea suport vector :  [1.15811087 0.79166269]
```

```
y_pred = svm_rbf.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("MSE : ", mse)
```

```
MSE :  3.941156748751104
```

Exercitiul 4 - 2025

Use the following code to generate a classification dataset:

```
X, y = make_classification(
    n_samples=1000,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_classes=2,
    n_clusters_per_class=1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the linear soft margin SVM algorithm with C = 100 for 100000 epochs on the train set.

Compute w, b, the second support vector, the accuracy on the test set, and the output probability distribution for the 8th sample in the test set.

```
rray([1.82388098]), array([2.1141893, -0.33704769]), 0.912, array([[0.22098076, 0.77901924]]))
```

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=2,
```

```
    n_informative=2,
    n_redundant=0,
    n_classes=2,
    n_clusters_per_class=1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
hyperparamC = 100
EPOCHS = 100000
```

```
svm_clf = SVC(C = hyperparamC,
              kernel = 'linear',
              max_iter = EPOCHS,
              random_state = 42,
              probability = True)

svm_clf.fit(X_train, y_train)
```

```
SVC
SVC(C=100, kernel='linear', max_iter=100000, probability=True, random_state=42)
```

```
w = svm_clf.coef_[0]
b = svm_clf.intercept_[0]
print("w : ", w)
print("b : ", b)
```

```
w :  [-0.94011825  2.42225277]
b :  1.8238809776394698
```

```
# support vectors
print("Al doilea suport vector : ", svm_clf.support_vectors_[2])
```

```
Al doilea suport vector :  [2.43473733 0.13841464]
```

```
# accuracy
accuracy = svm_clf.score(X_test, y_test)
print("Accuracy : ", accuracy)
```

```
Accuracy :  0.912
```

```
# the output probability distribution fot the 8th sample
y_pred = svm_clf.predict_proba(X_test)
print("Output probability distribution for the 8th sample in the test set : ", y_pred[7])
```

```
Output probability distribution for the 8th sample in the test set :  [0.22098076 0.7790192
```

Laborator 4

Exercitiul 1 - 1.jpg

Use the following code to generate a classification dataset:

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply a soft voting ensemble formed of a logistic regression algorithm, a random forest algorithm with 200 estimators, and a Gaussian RBF kernel SVM algorithm with γ = 1.5, on the train set.

Compute the accuracy on the test set and the output probability distribution of the 9th sample in the test set.

```
(0.912, array([[0.57070994, 0.21619792, 0.21309213]]))
```

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import accuracy_score
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
log_clf = LogisticRegression(solver="lbfgs", random_state=42)

rnd_clf = RandomForestClassifier(n_estimators = 200, random_state=42)

svm_clf = SVC(kernel = "rbf", gamma = 1.5, probability = True, random_state=42)

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
```

```
        voting='soft')

    voting_clf.fit(X_train, y_train)
```

```
VotingClassifier
lr: LogisticRegression
rf: RandomForestClassifier
svc: SVC
```

```
    for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)
        print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
```

```
LogisticRegression 0.784
RandomForestClassifier 0.916
SVC 0.916
VotingClassifier 0.912
```

```
    y_pred = voting_clf.predict(X_test)
    print("Accuracy : ", accuracy_score(y_test, y_pred))
```

```
Accuracy :  0.912
```

```
    # the output probability distribution of the 9th sample in the test set.

    y_pred = voting_clf.predict_proba(X_test)

    print("Output probability ", y_pred[8])
```

```
Output probability  [0.57072585 0.21619723 0.21307692]
```

Exercitiul 2 - 5.jpg

Use the following code to generate a classification dataset:

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply a gradient boosting ensemble algorithm formed of 200 decision trees, each tree having a maximum depth of 3, with a learning rate of 1.2 on the train set.

Compute the feature importances, the accuracy on the test set, and the output probability distribution of the 11th sample in the test set.

```
8, 0.34405552, 0.33884012]), 0.892, array([[8.55051214e-14, 2.46194816e-14, 1.00000000e+00]]))
```

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
gbct = GradientBoostingClassifier(n_estimators = 200,
                                  max_depth = 3,
                                  learning_rate = 1.2,
                                  random_state = 42)

gbct.fit(X_train, y_train)
```

```
GradientBoostingClassifier
GradientBoostingClassifier(learning_rate=1.2, n_estimators=200, random_state=42)
```

```
# Compute the feature importances
feature_importances = gbct.feature_importances_
print("Feature importances : ", feature_importances)
```

```
Feature importances :  [0.13183652 0.18539985 0.34488791 0.33787572]
```

```
y_pred = gbct.predict(X_test)

print("Accuracy : ", accuracy_score(y_test, y_pred))
```

```
Accuracy :  0.892
```

```
y_pred_proba = gbct.predict_proba(X_test)

print("Output prob 11: ", y_pred_proba[10])
```

```
Output prob 11:  [1.04008298e-13 2.99471001e-14 1.00000000e+00]
```

Exercitiul 3 - 7.jpg

Use the following code to generate a classification dataset:

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=2,
    n_clusters_per_class=1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the decision tree algorithm, with a minimum of 5 samples in each leaf on the train set.

Compute the feature importances, the accuracy on the test set, and the output probability distribution for the 9th sample in the test set.

```
821, 0.20425957, 0.35449143, 0.28619079]), 0.852, array([[0.33333333, 0.55555556, 0.11111111]])
```

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
tree_clf = DecisionTreeClassifier(min_samples_leaf = 5,
                                  random_state=42)
tree_clf.fit(X_train, y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
```

```
feature_importances = tree_clf.feature_importances_
print("Feature importances : ", feature_importances)
```

```
Feature importances :  [0.15805821 0.20428957 0.35449143 0.28316079]
```

```
accuracy = tree_clf.score(X_test, y_test)
print("Accuracy : ", accuracy)

y_pred = tree_clf.predict(X_test)
```

```
print("Accuracy", accuracy_score(y_test, y_pred))
```

```
Accuracy :  0.852
Accuracy 0.852
```

```
y_pred_proba = tree_clf.predict_proba(X_test)

print("Output prob 9: ", y_pred_proba[8])
```

```
Output prob 9:  [0.33333333 0.55555556 0.11111111]
```

Exercitiul 4 - 2025

Use the following code to generate a classification dataset:

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply a soft voting ensemble formed of a logistic regression algorithm, a random forest algorithm with 200 estimators, and a Gaussian RBF kernel SVM algorithm with γ = 1.5, on the train set.

Compute the accuracy on the test set and the output probability distribution of the 9th sample in the test set.

```
(0.912, array([[0.57070994, 0.21619792, 0.21309213]]))
```

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import accuracy_score
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
log_clf = LogisticRegression(solver="lbfgs", random_state=42)
rnd_clf = RandomForestClassifier(n_estimators=200, random_state=42)
svm_clf = SVC(kernel='rbf', gamma=1.5, probability=True, random_state=42)

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='soft')
voting_clf.fit(X_train, y_train)
```

```
VotingClassifier
lr: LogisticRegression
rf: RandomForestClassifier
svc: SVC
```

```
y_pred = voting_clf.predict(X_test)
print("Accuracy : ", accuracy_score(y_test, y_pred))

print("Accuracy : ", voting_clf.score(X_test, y_test))
```

```
Accuracy :  0.912
Accuracy :  0.912
```

```
y_pred_proba = voting_clf.predict_proba(X_test)

print("Output prob 9: ", y_pred_proba[8])
```

```
Output prob 9:  [0.57072585 0.21619723 0.21307692]
```

Laborator 5

Exercitiul 1 -> 2.jpg

Use the following code to generate a classification dataset:

```
X, y = make_circles(
    n_samples=1000,
    factor=0.1,
    noise=0.01,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the PCA algorithm with 2 components, on the train set.

Compute the components, the explained variance ratio, and the inverse transformation of the 11th sample in the test set.

```
[ 0.63888669, 0.76930085]]), array([0.50537804, 0.49462196]), array([-0.5139503, 0.82538319]))
```

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
```

```
X, y = make_circles(
    n_samples=1000,
    factor=0.1,
    noise=0.01,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X_train)
```

```
pca.components_
```

```
array([[ 0.76930085, -0.63888669],
       [ 0.63888669,  0.76930085]])
```

```
pca.explained_variance_ratio_
```

```
array([0.50537804, 0.49462196])
```

```
X_test_pca = pca.transform(X_test)
sample_11 = X_test_pca[10]
sample_11_inversed = pca.inverse_transform(sample_11)

sample_11_inversed
```

```
array([-0.5139503 ,  0.82538319])
```

Exercitiul 2 - 9.jpg

Use the following code to generate an S-shaped curve dataset:

```
X, y = make_s_curve(
    n_samples=1000,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the locally linear embedding algorithm with 3 components and 6 neighbors, on the train set.

Compute the transformation of the 11th sample in the test set.

```
array([-0.04161236, 0.04043414, 0.01098724])
```

```
from sklearn.datasets import make_s_curve
from sklearn.model_selection import train_test_split
from sklearn.manifold import LocallyLinearEmbedding
```

```
X, y = make_s_curve(
    n_samples=1000,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
lle = LocallyLinearEmbedding(n_components = 3,
                             n_neighbors = 6,
                             random_state = 42)

X_reduced = lle.fit_transform(X_train)
```

```
X_test_11 = lle.transform(X_test[10:11])
X_test_11
```

```
array([[ 0.04161253, -0.04042708,  0.01086122]])
```

Laborator 6

Exercitiul 1 -> 4.jpg

Use the following code to generate a classification dataset:

```
X, y = make_circles(
    n_samples=1000,
    factor=0.5,
    noise=0.1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the full K-means clustering algorithm with K = 5 and a maximum of 10 iterations on the train set.

Compute the label of the 5th sample in the test set, the third cluster center, the inertia on the train set, the negative inertia on the test set, and the silhouette score.

```
(0, array([-0.21019163, 0.67227348]), 104.42381022725004, -33.16508490326328, 0.08178833442790
```

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
X, y = make_circles(
    n_samples=1000,
    factor=0.5,
    noise=0.1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
kmeans_iter = KMeans(n_clusters = 5,
                     init = "random",
                     algorithm = "lloyd",
                     max_iter = 10,
                     random_state = 42)

kmeans_iter.fit(X_train)
```

```
KMeans
KMeans(init='random', max_iter=10, n_clusters=5, random_state=42)
```

```
y_pred = kmeans_iter.predict(X_test)
print("Label 5: ", y_pred[4])
```

```
Label 5:  2
```

```
print("Third cluster center : ", kmeans_iter.cluster_centers_[2])
```

```
Third cluster center :  [-0.16050603 -0.72361306]
```

```
print("Inertia : ", kmeans_iter.inertia_)
```

```
Inertia :  104.44216577761102
```

```
print("Negative inertia : ", kmeans_iter.score(X_test))
```

```
Negative inertia :  -33.19362229769065
```

```
silhouette_score(X_train, kmeans_iter.labels_)
```

```
np.float64(0.35665508838880466)
```

Exercitiul 2 -> 6.jpg

Use the following code to generate a classification dataset:

```
X, y = make_circles(
    n_samples=1000,
```

```
    factor=0.6,
    noise=0.1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the K-means clustering algorithm with K = 3 on the train set.

Compute the label of the 11th sample in the test set, the first cluster center, the inertia on the train set, the negative inertia on the test set, and the normalized mutual information score.

```
(2, array([0.24530604, 0.53353481]), 186.75691886796457, -58.722482875379455, 0.002387525566746
```

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.metrics.cluster import normalized_mutual_info_score
```

```
X, y = make_circles(
    n_samples=1000,
    factor=0.6,
    noise=0.1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
kmeans_iter = KMeans(n_clusters = 3,
                     init = "random",
                     random_state = 42)

kmeans_iter.fit(X_train)
```

```
▾ KMeans
KMeans(init='random', n_clusters=3, random_state=42)
```

```
y_pred = kmeans_iter.predict(X_test)
print("Label 11: ", y_pred[10])
```

```
Label 11:  2
```

```
print("The first cluster center : ", kmeans_iter.cluster_centers_[0])
```

```
The first cluster center :  [-0.28055464 -0.60136566]
```

```
print("Inertia : ", kmeans_iter.inertia_)
```

```
Inertia :  187.87461196970017
```

```
print("Negative inertia : ", kmeans_iter.score(X_test))
```

```
Negative inertia :  -62.528974297071805
```

```
print("Normalized mutual info score : ", normalized_mutual_info_score(y_test, y_pred))
```

```
Normalized mutual info score :  0.0006108018151815857
```

Exercitiul 3 - 8.jpg

Use the following code to generate a classification dataset:

```
X, y = make_circles(
    n_samples=1000,
    factor=0.5,
    noise=0.1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the K-means clustering algorithm with K = 4, random initialization, and 15 consecutive initializations, on the train set.

Compute the label of the 9th sample in the test set, the second cluster center, the inertia on the train set, the negative inertia on the test set, and the adjusted Rand score.

```
l, array([-0.6731441, -0.07718862]), 132.22416031964735, -40.94325371413339, -0.003124727742959
```

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.metrics.cluster import adjusted_rand_score
```

```
X, y = make_circles(
    n_samples=1000,
    factor=0.5,
    noise=0.1,
    random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
kmeans_iter = KMeans(n_clusters = 4,
                     init = "random",
                     n_init = 15,
                     random_state = 42)

kmeans_iter.fit(X_train)
```

```
KMeans
KMeans(init='random', n_clusters=4, n_init=15, random_state=42)
```

```
y_pred = kmeans_iter.predict(X_test)
print(y_pred[8])
```

```
1
```

```
kmeans_iter.cluster_centers_[1]
```

```
array([-0.72902152, -0.0517824 ])
```

```
kmeans_iter.inertia_
```

```
126.40757091483536
```

```
kmeans_iter.score(X_test)
```

```
-39.531936561952456
```

```
adjusted_rand_score(y_test, y_pred)
```

```
-0.003316434413645542
```

Exercitiul 4 - 2025

Use the following code to generate a classification dataset:

```
X, y = make_circles(
    n_samples=1000,
    factor=0.5,
    noise=0.1,
    random_state=42,
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the mini-batch K-means clustering algorithm with K = 3 on the train set.

Compute the label of the 8th sample in the test set, the first cluster center, the inertia on the train set, the negative inertia on the test set, and the adjusted Rand score.

```
(1, array([-0.47651268, -0.41297484]), 188.93622125753416, -59.56111648796079, -0.004755512475
```

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.cluster import MiniBatchKMeans
from sklearn.metrics import silhouette_score
from sklearn.metrics.cluster import adjusted_rand_score
```

```
X, y = make_circles(
    n_samples=1000,
```

```
        factor=0.5,
        noise=0.1,
        random_state=42,
    )
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size = 0.25,
                                                    random_state = 42)
```

```
minibatch_kmeans = MiniBatchKMeans(n_clusters = 3,
                                   random_state=42)
minibatch_kmeans.fit(X_train)
```

```
▾ MiniBatchKMeans
MiniBatchKMeans(n_clusters=3, random_state=42)
```

```
y_pred = minibatch_kmeans.predict(X_test)
print(y_pred[7])
```

```
0
```

```
minibatch_kmeans.cluster_centers_[0]
```

```
array([0.6333653 , 0.04548053])
```

```
minibatch_kmeans.inertia_
```

```
187.1333213572188
```

```
minibatch_kmeans.score(X_test)
```

```
-60.457095159727615
```

```
adjusted_rand_score(y_test, y_pred)
```

```
-0.004702970229508548
```

Exercitiul 5 - 2024

Use the following code to generate a classification dataset:

```
X, y = make_circles(
    n_samples=1000,
    factor=0.5,
    noise=0.1,
    random_state=42,
)
```

Apply the agglomerative clustering algorithm with 5 clusters and single linkage on this dataset.

Compute the label of the 5th sample in the dataset and the normalized mutual information score.

```
(0, 0.007673110475351361)
```

```
from sklearn.datasets import make_circles
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics.cluster import normalized_mutual_info_score
```

```
X, y = make_circles(
    n_samples=1000,
    factor=0.5,
    noise=0.1,
    random_state=42,
)
```

```
agg = AgglomerativeClustering(n_clusters=5,
                              linkage="single")
clusters = agg.fit_predict(X)
```

```
print("The label of the 5th sample : ", clusters[4])
```

```
The label of the 5th sample :  0
```

```
normalized_mutual_info_score(y, clusters)
```

```
np.float64(0.007673110475351361)
```